Banco Orbital

Documentación técnica

Cómo está construido Banco Orbital, parte por parte.

Octubre de 2026 · orbital.net.ar · Máximo Bertaina y Gonzalo Castellano

1. El sistema de un vistazo

Cinco repositorios, cinco contenedores, una base de datos que vive afuera.

```
internet ──→ Cloudflare ──→ :443 nginx   (el único contenedor expuesto)
                            ├── app.orbital.net.ar
                            │     ├── /            → frontend  (React compilado)
                            │     ├── /api  /auth  → backend:3001
                            │     └── /webhooks/   → backend:3001   (Clerk)
                            └── orbital.net.ar     → landing

backend ──→ proveedores:4000   (red interna, sin salida a internet)
backend ──→ Supabase           (PostgreSQL 17, 22 tablas)
backend ──→ Banco Central      (transferencias entre bancos)
backend ──→ DolarAPI           (cotización del dólar)
backend ──→ Gemini             (el asistente)
```

Repositorio	Qué es	Stack	Líneas
`banco-backend`	La API y toda la lógica del banco	Node 22 · Express 5 · PostgreSQL	~12.000
`banco-frontend`	El home banking	React 18 · Vite 6 · Tailwind v4	~16.300
`banco-landing`	El sitio público	React 18 · Vite 6 · Tailwind v4	~1.300
`banco-proveedores`	Mock de servicios externos	Node · Express	~450
`banco-infra`	Contenedores, nginx, despliegue	Docker Compose · nginx · certbot	—

El portal y la API viven **en el mismo origen** (`app.orbital.net.ar`). Eso elimina el CORS del navegador y hace que el token de Clerk viaje sin fricción.

---

2. El backend

Cómo está organizado

```
src/
├── app.js            Express: helmet, CORS, rate limit, logs, montaje de rutas
├── server.js         Arranque, tareas de fondo y apagado ordenado
├── config/env.js     Lee y valida las variables de entorno
├── db/pool.js        El pool de conexiones a PostgreSQL
├── routes/index.js   Monta cada router bajo su prefijo
├── middlewares/      Autenticación, validación, idempotencia, errores
├── modules/          Un par router + service por dominio
└── utils/            Dinero, errores HTTP, auditoría, logger
```

La regla que ordena todo el backend: **el router no sabe SQL y el service no sabe de HTTP.**

- El **router** valida la entrada con zod, comprueba el rol y traduce el resultado a una respuesta. No toca la base.
- El **service** tiene las reglas del negocio y las transacciones. No sabe qué es un `req` ni un `res`; cuando algo sale mal lanza un `HttpError` y el middleware de errores lo convierte.

Los services se construyen con una fábrica que recibe sus dependencias (`createXService({ pool, logger, ... })`). Eso es lo que permite probarlos con un pool de mentira, sin levantar una base.

Los módulos

Módulo	De qué se ocupa
`auth`	Login, registro y perfil contra Clerk
`transacciones`	Transferencias, depósitos, extracciones y cambio de divisa
`cuentas`	Apertura de cuentas, saldos y movimientos
`tarjetas`	Emisión, niveles, autorizaciones de consumo y resumen
`prestamos`	Simulación, otorgamiento, cuotas, mora y aprobación del gerente
`plazos-fijos`	Constitución, acreditación y barrido de vencimientos
`servicios` / `recargas`	Pagos contra `banco-proveedores`
`cobros`	Los QR: generarlos y pagarlos
`central-bank`	Todo lo que va y viene del Banco Central
`chatbot`	El asistente, con Gemini y herramientas de solo lectura
`bonificaciones`	La órbita secreta: US$ 5 de bienvenida

Módulo	De qué se ocupa
`crud-router`	CRUD genérico para las tablas de catálogo

La API

90 rutas declaradas en los módulos, más el CRUD genérico sobre nueve entidades (`personas`, `roles`, `usuarios`, `cuentas`, `destinatarios`, `auditoria`, `personas_roles`, `tipos_cuenta`, `tipos_transaccion`), que suma cinco rutas por cada una.

El contrato completo está en `banco-backend/docs/openapi-banco-orbital.yaml`. Lo más usado:

Identidad — `/auth`

```
POST   /auth/login            PUT   /auth/perfil
POST   /auth/register         PATCH /auth/perfil
GET    /auth/perfil           POST  /auth/logout
```

Plata

```
POST   /api/transacciones/transferir     POST  /api/transacciones/deposito
POST   /api/transacciones/cambio         POST  /api/transacciones/extraccion
GET    /api/transacciones                GET   /api/transacciones/:id
GET    /api/transacciones/destinatario/resolver
```

Cuentas y movimientos

```
GET    /api/personas/:id/full            la foto completa del cliente
POST   /api/personas/:id/cuentas/apertura
GET    /api/cuentas/:id/movimientos      GET  /api/cuentas/:id/resumen-gastos
GET    /api/cuentas/:id/movimientos/exportar     (CSV)
PUT    /api/cuentas/:id/alias
```

Productos

```
POST   /api/tarjetas                     GET  /api/personas/:id/tarjetas/oferta
POST   /api/tarjetas/:id/autorizaciones  POST /api/tarjetas/:id/bloqueo
POST   /api/prestamos                    POST /api/prestamos/simulaciones
GET    /api/prestamos/pendientes         POST /api/prestamos/:id/aprobacion
POST   /api/prestamos/:id/pagos          POST /api/prestamos/:id/rechazo
POST   /api/plazos-fijos                 POST /api/plazos-fijos/simulaciones
```

Cobros por QR — montados aparte en `app.js`, con su propio middleware de Clerk

```
POST   /api/cobros          generar un QR (vive 10 minutos)
GET    /api/cobros/:id      ver qué se va a pagar
POST   /api/transferencias  pagarlo (exige Idempotency-Key y reverificación)
```

Banco Central — 24 rutas: registro del banco, personas, CBU y alias, transferencias entrantes y salientes, sincronización de cuentas y consulta a la Central de Deudores.

Los middlewares

Middleware	Qué hace
`clerk-auth`	Valida el token y deja el usuario en `req.usuarioActual`
`require-active-user`	Comprueba que el usuario exista y esté activo en la base
`require-complete-profile`	Frena a quien no terminó el alta
`validate`	Valida y **reemplaza** body, params o query con el valor ya convertido
`idempotency`	Guarda la respuesta por clave; si llega repetida, la devuelve sin re-ejecutar
`error-handler`	Traduce `HttpError` a JSON; lo inesperado sale como 500 sin filtrar detalles

> **Una trampa de Express 5.** `req.query` es una propiedad de sólo lectura. Asignarle no lanza error, pero tampoco hace nada, y el handler sigue recibiendo los strings crudos de la URL. Eso dejaba sin efecto toda la paginación: pedir `?limit=100` devolvía 20 igual. Por eso `validate` **redefine** la propiedad en vez de asignarla.

Las tareas de fondo

Arrancan en `server.js`, no en `app.js`, para que los tests que importan la app no disparen timers. Las dos se apagan con `SINCRONIZAR_ENTRANTES=false`.

- **Sincronizador de entrantes**, cada 15 minutos. El Banco Central no avisa cuando llega plata: hay que preguntarle, y su endpoint sólo mira 24 horas atrás. Pregunta por los últimos 30 minutos, con ventanas superpuestas a propósito: un índice único hace que repetir sea seguro.
- **Vencedor de cobros**, cada 10 minutos. Marca `vencido` los QR a los que se les pasó la hora. No es lo que impide pagarlos —eso lo hace el endpoint de pago— sino lo que mantiene la base diciendo la verdad.

Apagado ordenado

Docker manda `SIGTERM` y espera 10 segundos. Sin esto, un `docker compose restart` corta la conexión en medio de una transferencia y el cliente ve un error de red sin saber si la plata se movió. Con esto, el server deja de aceptar conexiones nuevas, termina las que están en curso y recién ahí cierra el pool. A los 8 segundos sale igual: mejor cortar que esperar a que Docker mate el proceso.

---

3. La base de datos

PostgreSQL 17 en Supabase, **22 tablas**. No corre en el servidor: es un servicio aparte. Prisma está en el repo como fuente de verdad del esquema, pero **la aplicación no lo usa en runtime**: todas las consultas son SQL con `pg`.

Las migraciones no las corre ningún contenedor. Se aplican a mano con `psql`. Es la forma más difícil de romper la base por accidente.

Tabla	Para qué
`personas`	La persona física: nombre, DNI, correo, teléfono
`usuarios`	La identidad de acceso, atada a Clerk por `clerk_id`
`personas_roles` · `roles`	Quién es qué dentro del banco
`cuentas` · `tipos_cuenta`	Cuentas con CBU, alias, moneda, saldo y límite
`transacciones` · `tipos_transaccion`	Todo movimiento de plata
`destinatarios`	La agenda de cada cliente
`tarjetas` · `autorizaciones`	Débito y crédito, y cada consumo autorizado
`prestamos` · `cuotas_prestamo`	Préstamos y su plan de pagos
`plazos_fijos`	Inversiones a plazo
`cobros`	Los QR de cobro
`bonificaciones`	La órbita secreta
`auditoria`	Quién hizo qué, cuándo y desde qué IP
`idempotency_keys`	Para que una operación no se ejecute dos veces
`banco_central_configuracion` · `banco_central_registro`	Credenciales y registro del banco

> Las credenciales del Banco Central viven **en la base**, en `banco_central_configuracion`, no en el `.env`. Las variables `CENTRAL_BANK_*` del servidor están vacías y son un vestigio.

4. El frontend

Cómo está organizado

```
src/
├── app/           providers, router y App
├── pages/         PortalPage y su configuración de secciones
├── features/      una carpeta por dominio
│   └── <dominio>/
│       ├── api/         las llamadas de ese dominio
│       ├── sections/    la pantalla que se monta en el portal
│       └── components/ lo que usa esa pantalla
├── components/    lo compartido: Header, tarjetas, ui/ (shadcn)
├── hooks/         usePortalPage y compañía
└── lib/
    ├── api/       el cliente HTTP: token, idempotencia, errores
    ├── queries/   los hooks de TanStack Query
    ├── schemas/   validación con zod
    └── utils/     plata, fechas, formato
```

Quince features: `admin`, `auth`, `bonificaciones`, `cambio`, `chatbot`, `cuentas`, `dashboard`, `destinatarios`, `inversiones`, `pagos`, `personas`, `prestamos`, `qr`, `tarjetas`, `transacciones`.

Cómo se maneja el estado

- **TanStack Query** para todo lo que viene del servidor. Las claves están centralizadas en `lib/queries/keys.ts`, y cada mutation que mueve plata invalida `personas.full` y `transacciones` al terminar: el saldo y los movimientos se actualizan solos sin que la pantalla tenga que acordarse.
- `useState` **local** para lo que es de una pantalla y no le importa a nadie más.
- No hay Redux ni store global. No hizo falta.

> **Dónde vive una mutation.** Si al terminar desaparece la fila que la disparó, la mutation tiene que vivir en la sección, no en la tarjeta. React Query descarta los callbacks de un componente desmontado: desde la tarjeta, el aviso de "listo" nunca llegaba a mostrarse y la fila desaparecía sin decir nada.

Secciones del portal

Las que ve cada quien dependen del rol activo: `Resumen`, `Cuentas`, `Transferir`, `Pagos`, `Dólares`, `Tarjetas`, `Préstamos`, `Inversiones`, `Destinatarios`, más `Solicitudes` (gerente y admin) y `Admin` (sólo admin).

Las secciones se cargan por separado (code splitting), con el isotipo girando de fallback.

Rutas

```
/            el portal
/ingresar    /registro      Clerk, con la marca de Orbital
/cobrar      generar un QR
/escanear    leer un QR con la cámara
/pagar?id=   confirmar el pago
```

En el teléfono

- **Los blancos de toque son de 44px cuando la pantalla se maneja con el dedo**, con una variante `dedo:` que envuelve `@media (pointer: coarse)`. La condición no puede ser el ancho: un teléfono dado vuelta mide 844px y una tablet 768, y con un corte por ancho los dos volvían al tamaño de escritorio aunque se sigan tocando con el dedo.
- La fila de secciones se desliza, con una sombra del lado donde queda contenido.
- Es una **PWA**: se instala desde el navegador, sin pasar por ninguna tienda.

---

5. El mock de proveedores

Un Express chico que simula las empresas de servicios y las operadoras de celular. Vive en la red interna de Docker y **no se puede alcanzar desde internet**. Pide una API key.

```
GET   /health
GET   /empresas                        listar, con filtro por rubro
GET   /empresas/:empresaId/deuda       qué debe un cliente
POST  /empresas/:empresaId/pagos       pagar
GET   /operadoras                      listar
POST  /operadoras/:operadoraId/recargas
```

Tiene su propio middleware de idempotencia: pagar dos veces la misma factura con la misma clave devuelve el mismo comprobante.

---

6. La landing

React con Vite y los mismos componentes del portal, para que se vean de la misma familia. Todo el texto vive en `src/datos/contenido.ts`, separado de los componentes: cambiar una cifra no obliga a tocar JSX.

Secciones: `Hero`, `Productos`, `Plastico`, `Niveles`, `Beneficios`, `Instalacion`, `ComoEmpezar`, `Preguntas`, `Cierre`.

---

7. La infraestructura

Un `docker-compose.yml` con cinco servicios más certbot. **Sólo nginx publica puertos** (80 y 443); el resto vive en la red interna `interna`.

Servicio	Imagen	Puerto
`proxy`	nginx:alpine	80, 443 (los únicos expuestos)
`certbot`	certbot/certbot	—
`backend`	construida del repo	3001 interno
`frontend`	construida (Vite → nginx)	80 interno
`landing`	construida (Vite → nginx)	80 interno
`proveedores`	construida del repo	4000 interno

Todos con `restart: unless-stopped` y rotación de logs: en un disco chico, los logs sin rotar se lo comen.

Decisiones que conviene conocer

- **`node:22-slim`, no `alpine`.** El formateo de plata y fechas usa `Intl` con locale `es-AR`. Alpine trae un ICU recortado y eso devuelve strings distintos **en silencio**, que es la peor forma de fallar.
- **`TZ=America/Argentina/Buenos_Aires`.** Sin esto el contenedor corre en UTC y un préstamo pedido a las 22:00 aparece fechado al día siguiente.
- **`trust proxy`.** Detrás de nginx, sin esta línea el rate limit cuenta a todos los usuarios como una sola IP y la auditoría registra la IP interna de Docker.
- **nginx no cachea** `sw.js`, `registerSW.js`, `manifest.webmanifest` ni `index.html`. Un service worker viejo que queda pegado sigue sirviendo la versión anterior y desde el servidor no hay forma de sacárselo.
- **Cloudflare adelante**, con `set_real_ip_from` sobre sus rangos para que la IP del cliente llegue de verdad.

Los comandos

```
cd /opt/orbital/banco-infra

./scripts/deploy.sh              # traer cambios y actualizar
./scripts/deploy.sh backend      # sólo un servicio
./scripts/respaldo.sh            # un dump comprimido de la base
./scripts/certificados.sh        # emitir o renovar certificados

docker compose ps                # estado
docker compose logs -f backend   # mirar la API
```

> **El build ahoga la máquina.** Compilar el portal en 1 GB consume toda la memoria mientras dura. Si justo después no se puede entrar por SSH, no es la clave ni el servidor caído: hay que esperar dos minutos.

---

8. Seguridad

Autenticación	Clerk. El backend valida el token en cada request, nunca confía en el cliente
Autorización	Seis roles. Repartirlos es exclusivo de `admin`; `auditor` sólo lee
Auditoría	Append-only: usuario, acción, entidad, antes, después, IP y origen
Idempotencia	Obligatoria en todo lo que mueve plata
Concurrencia	`SELECT … FOR UPDATE` en orden fijo por id, para no deadlockear
Saldos	`UPDATE … WHERE saldo >= $1`: no hay ventana entre comprobar y descontar
Límites	`limite_transferencia` por tipo de cuenta, en el formulario y en el QR
Rate limit	Global por IP, más uno propio y más estricto en `/auth` y en los cobros
Cabeceras	`helmet` en la API
Secretos	En `.env` creados a mano en el servidor con `chmod 600`. Nunca se commitean

Los roles

Rol	Qué puede hacer
`cliente`	Operar con lo suyo, y nada más
`operador`	Alta y gestión de clientes y cuentas
`tesoreria`	Operaciones internas de caja
`gerente`	Aprobar o rechazar préstamos que el filtro automático no resolvió
`auditor`	Leer la auditoría; no puede modificar nada
`admin`	Todo, incluido repartir roles

El gerente es el único que puede aprobar un préstamo a alguien con mala situación crediticia, y para hacerlo **tiene que escribir un motivo de al menos diez caracteres** que queda registrado con su nombre.

---

9. Los tests

414 en total: 386 en el backend (33 archivos) y 28 en el portal (3 archivos). Corren con Vitest.

```
cd banco-backend  && npx vitest run
cd banco-frontend && npx vitest run
```

Lo que se cubre en el backend: el value object `Dinero`, el cálculo financiero de préstamos y plazos fijos, el riesgo crediticio, los niveles de tarjeta, las transacciones, la sincronización de entrantes, el circuito de aprobación del gerente, el pago por QR, el chatbot y el middleware de validación.

En el portal: el formato de montos, el botón de compra y venta máxima, y la bandeja del gerente de punta a punta.

> **La pregunta que se le hace a cada test nuevo: si rompo el código a propósito, ¿se da cuenta?** Cada tanda se verifica saboteando el código —sacarle un `trim`, invertir un booleano, bajar un límite— y comprobando que los tests fallen. Un test que pasa igual con el código roto no prueba nada.

---

10. Levantarlo en tu máquina

```
# Backend
cd banco-backend && npm install && npm run dev        # :3001

# Mock de proveedores
cd banco-proveedores && npm install && npm run dev    # :4000

# Portal
cd banco-frontend && npm install && npm run dev       # :5173

# Landing
cd banco-landing && npm install && npm run dev        # :5174
```

Hace falta un `.env` en cada uno; los `.env.example` listan las variables.

> **Node 20 o superior.** Con Node 18 el build del portal falla con `crypto is not defined`, porque el plugin de PWA usa el `crypto` global.
> **Ojo: la base es la de producción.** No hay una de desarrollo aparte. Lo que toques en local se toca en serio.

También se puede levantar todo junto en Docker, sin dominio ni certificados:

```
cd banco-infra && docker compose -f docker-compose.local.yml up --build
```

Portal en `http://localhost:8080`.

---

11. Dónde está cada cosa

Documento	Qué hay adentro
`banco-infra/docs/documentacion-del-proyecto.md`	La historia: cómo arrancó y cómo se trabaja
`banco-infra/docs/manual-de-despliegue.pdf`	Montar el servidor de cero, paso a paso
`banco-infra/README.md`	El día a día del servidor, respaldos, qué mirar cuando falla
`banco-backend/README.md`	Setup y arquitectura de la API
`banco-backend/docs/openapi-banco-orbital.yaml`	El contrato completo
`banco-backend/docs/GLOSARIO.md`	Cómo se llama cada cosa y por qué
`banco-backend/docs/PLAN.md`	Las fases y las decisiones tomadas
